dff9-W4111-Spring-2026 -HW3.ipynb

Introduction

Homework Overview

There is one version of the homework that both the programming and non-programming tracks complete.

There are 4 categories of question:

1. Short answer questions testing general knowledge and the ability to reason about the course material.
2. SQL questions testing the ability to use SQL to produce data results from descriptions of the desired output.
3. Questions testing the ability to use MongoDB to produce results from descriptions of the desired output.
4. Questions testing the ability to use Neo4j to produce results from the descriptions of the desired output.

Submission Instructions

The homework is due at 11:59 PM on Sunday, 18-April-2026.

Please see Ed discussion for detailed submission instructions and for clarifications and answering questions.

Part 0 — Setup

iPython-SQL

In [1]:
```
%load_ext sql
```

Set the proper root user ID and password for MySQL in the python cell below.

In [2]:
```
mysql_root_user = 'root'
mysql_root_password = 'david-database'

mysql_url = f"mysql+pymysql://{mysql_root_user}:{mysql_root_password}@localhost"
%pip install cryptography
```

```
Requirement already satisfied: cryptography in /Users/davidnguyen/PycharmProjects/W4111-Intro-to-Databases-
Spring-2026/venv/lib/python3.9/site-packages (46.0.7)
Requirement already satisfied: cffi>=2.0.0 in /Users/davidnguyen/PycharmProjects/W4111-Intro-to-Databases-S
pring-2026/venv/lib/python3.9/site-packages (from cryptography) (2.0.0)
Requirement already satisfied: typing-extensions>=4.13.2 in /Users/davidnguyen/PycharmProjects/W4111-Intro-
to-Databases-Spring-2026/venv/lib/python3.9/site-packages (from cryptography) (4.15.0)
Requirement already satisfied: pycparser in /Users/davidnguyen/PycharmProjects/W4111-Intro-to-Databases-Spr
ing-2026/venv/lib/python3.9/site-packages (from cffi>=2.0.0->cryptography) (2.23)

[notice] A new release of pip is available: 23.2.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

In [3]:
```
mysql_url
```

Out[3]:
```
'mysql+pymysql://root:david-database@localhost'
```

In [4]:
```
%sql $mysql_url
```

In [5]:
```
%config SqlMagic.style = '_DEPRECATED_DEFAULT'
```

In [6]:
```
%sql select * from db_book.student where dept_name="Comp. Sci."
```

```
 * mysql+pymysql://root:***@localhost
4 rows affected.
```

Out[6]:

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
54321	Williams	Comp. Sci.	54
76543	Brown	Comp. Sci.	58

SQL Alchemy

In [7]:
```
from sqlalchemy import create_engine, text
```

In [8]:
```
engine = create_engine(mysql_url)
```

In [9]:
```
# Raw SQL query
sql = text("SELECT * FROM db_book.student WHERE dept_name = :dept")

# Execute query
with engine.connect() as connection:
    result = connection.execute(sql, {"dept": "Comp. Sci."})

    for row in result:
        print(row)
```

```
('00128', 'Zhang', 'Comp. Sci.', Decimal('102'))
('12345', 'Shankar', 'Comp. Sci.', Decimal('32'))
('54321', 'Williams', 'Comp. Sci.', Decimal('54'))
('76543', 'Brown', 'Comp. Sci.', Decimal('58'))
```

Pandas

In [10]:
```
import pandas
```

In [11]:
```
result = pandas.read_sql(con=engine, sql="select * from db_book.student where dept_name='Comp. Sci.'")
```

In [12]:
```
result
```

Out[12]:

	ID	name	dept_name	tot_cred
0	00128	Zhang	Comp. Sci.	102.0
1	12345	Shankar	Comp. Sci.	32.0
2	54321	Williams	Comp. Sci.	54.0
3	76543	Brown	Comp. Sci.	58.0

PyMySQL

In [13]:
```
import pymysql
```

In [14]:
```
conn = pymysql.connect(
    host="localhost",
    user=mysql_root_user,
    password=mysql_root_password,
    cursorclass=pymysql.cursors.DictCursor,
    autocommit=True
)
```

In [15]:
```
cursor = conn.cursor()
```

In [16]:
```
result = cursor.execute("select * from db_book.student where dept_name='Comp. Sci.'")
result = cursor.fetchall()
```

In [17]:
```
result
```

Out[17]:
```
[{'ID': '00128',
  'name': 'Zhang',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('102')},
 {'ID': '12345',
  'name': 'Shankar',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('32')},
 {'ID': '54321',
  'name': 'Williams',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('54')},
 {'ID': '76543',
  'name': 'Brown',
  'dept_name': 'Comp. Sci.',
  'tot_cred': Decimal('58')}]
```

MongoDB

Test Connectivity

In [101…
```python
import pymongo
```

In [102…
```python
from pymongo import MongoClient, errors
```

In [103…
```python
import json
```

In [104…
```python
#
# You will need to change this URL to connect to the MongoDB instance you created in HW3 Setup
#
#mongourl = "mongodb+srv://dbuser:sBJ1btaxRAWiqfm1@cluster0.t8qdk.mongodb.net/?appName=Cluster0"
mongourl = "mongodb+srv://gannetslumper2y_db_user:pfRAm13a80kHKZgT@cluster0.zpny5ca.mongodb.net/?appName=C
```

In [105…
```python
client = MongoClient(mongourl)
```

In [106…
```python
#
# Your answer wrt to databases and collections will be different based on the databases
# and collections you previously created. You connected successfully if you did not get an error.
#

try:
    # Create a MongoClient with a short timeout for quick failure if unreachable
    # client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=5000)

    # Attempt to retrieve server information (forces a connection call)
    server_info = client.server_info()
    print("✅ Successfully connected to MongoDB Atlas!")
    print("Server Info:", server_info.get("version", "Unknown version"))

    # Optional: List available databases
    print("\nDatabases on this cluster:")
    for db_name in client.list_database_names():
        print(" -", db_name)

except errors.ServerSelectionTimeoutError as err:
    print("❌ Could not connect to MongoDB Atlas. Timeout occurred.")
    print("Error details:", err)
except errors.OperationFailure as err:
    print("❌ Authentication failed. Check your username/password.")
    print("Error details:", err)
except Exception as err:
    print("❌ An unexpected error occurred.")
    print("Error details:", err)
```

```
✅ Successfully connected to MongoDB Atlas!
Server Info: 8.0.21

Databases on this cluster:
 - admin
 - local
```

Load Classicmodels

In [124…
```python
import json
```

In [125…
```python
with open('customers.json', 'r') as in_file:
    customers = json.load(in_file)
```

In [126…
```python
customers[0:2]
```

Out[126…
```
[{'customerNumber': 103,
  'customerName': 'Atelier graphique',
  'contact': {'lastName': 'Schmitt', 'firstName': 'Carine '},
  'phone': '40.32.2555',
  'address': {'addressLine1': '54, rue Royale',
   'addressLine2': None,
   'city': 'Nantes',
   'state': None,
   'country': 'France',
   'postalCode': '44000'},
  'salesRepNumber': 1370,
  'creditLimit': 21000.0},
 {'customerNumber': 112,
  'customerName': 'Signal Gift Stores',
  'contact': {'lastName': 'King', 'firstName': 'Jean'},
  'phone': '7025551838',
  'address': {'addressLine1': '8489 Strong St.',
   'addressLine2': None,
   'city': 'Las Vegas',
   'state': 'NV',
   'country': 'USA',
   'postalCode': '83030'},
  'salesRepNumber': 1166,
  'creditLimit': 71800.0}]
```

In [127…
```
len(customers)
```

Out[127…
```
122
```

In [111…
```
#BE CAREFUL. DO NOT LOAD THIS MULTIPLE TIMES. THE LENGTH SHOULD BE 122 IN MongdDB Cloud Atlas.
client['classicmodels']['customers'].insert_many(customers)
```

Out[111…
```
InsertManyResult([ObjectId('69e822d1cfc909fc9ae66847'), ObjectId('69e822d1cfc909fc9ae66848'), ObjectId('69
e822d1cfc909fc9ae66849'), ObjectId('69e822d1cfc909fc9ae6684a'), ObjectId('69e822d1cfc909fc9ae6684b'), Obje
ctId('69e822d1cfc909fc9ae6684c'), ObjectId('69e822d1cfc909fc9ae6684d'), ObjectId('69e822d1cfc909fc9ae6684
e'), ObjectId('69e822d1cfc909fc9ae6684f'), ObjectId('69e822d1cfc909fc9ae66850'), ObjectId('69e822d1cfc909f
c9ae66851'), ObjectId('69e822d1cfc909fc9ae66852'), ObjectId('69e822d1cfc909fc9ae66853'), ObjectId('69e822d
1cfc909fc9ae66854'), ObjectId('69e822d1cfc909fc9ae66855'), ObjectId('69e822d1cfc909fc9ae66856'), ObjectId
('69e822d1cfc909fc9ae66857'), ObjectId('69e822d1cfc909fc9ae66858'), ObjectId('69e822d1cfc909fc9ae66859'),
ObjectId('69e822d1cfc909fc9ae6685a'), ObjectId('69e822d1cfc909fc9ae6685b'), ObjectId('69e822d1cfc909fc9ae6
685c'), ObjectId('69e822d1cfc909fc9ae6685d'), ObjectId('69e822d1cfc909fc9ae6685e'), ObjectId('69e822d1cfc9
09fc9ae6685f'), ObjectId('69e822d1cfc909fc9ae66860'), ObjectId('69e822d1cfc909fc9ae66861'), ObjectId('69e8
22d1cfc909fc9ae66862'), ObjectId('69e822d1cfc909fc9ae66863'), ObjectId('69e822d1cfc909fc9ae66864'), Object
Id('69e822d1cfc909fc9ae66865'), ObjectId('69e822d1cfc909fc9ae66866'), ObjectId('69e822d1cfc909fc9ae6686
7'), ObjectId('69e822d1cfc909fc9ae66868'), ObjectId('69e822d1cfc909fc9ae66869'), ObjectId('69e822d1cfc909f
c9ae6686a'), ObjectId('69e822d1cfc909fc9ae6686b'), ObjectId('69e822d1cfc909fc9ae6686c'), ObjectId('69e822d
1cfc909fc9ae6686d'), ObjectId('69e822d1cfc909fc9ae6686e'), ObjectId('69e822d1cfc909fc9ae6686f'), ObjectId
('69e822d1cfc909fc9ae66870'), ObjectId('69e822d1cfc909fc9ae66871'), ObjectId('69e822d1cfc909fc9ae66872'),
ObjectId('69e822d1cfc909fc9ae66873'), ObjectId('69e822d1cfc909fc9ae66874'), ObjectId('69e822d1cfc909fc9ae6
6875'), ObjectId('69e822d1cfc909fc9ae66876'), ObjectId('69e822d1cfc909fc9ae66877'), ObjectId('69e822d1cfc9
09fc9ae66878'), ObjectId('69e822d1cfc909fc9ae66879'), ObjectId('69e822d1cfc909fc9ae6687a'), ObjectId('69e8
22d1cfc909fc9ae6687b'), ObjectId('69e822d1cfc909fc9ae6687c'), ObjectId('69e822d1cfc909fc9ae6687d'), Object
Id('69e822d1cfc909fc9ae6687e'), ObjectId('69e822d1cfc909fc9ae6687f'), ObjectId('69e822d1cfc909fc9ae6688
0'), ObjectId('69e822d1cfc909fc9ae66881'), ObjectId('69e822d1cfc909fc9ae66882'), ObjectId('69e822d1cfc909f
c9ae66883'), ObjectId('69e822d1cfc909fc9ae66884'), ObjectId('69e822d1cfc909fc9ae66885'), ObjectId('69e822d
1cfc909fc9ae66886'), ObjectId('69e822d1cfc909fc9ae66887'), ObjectId('69e822d1cfc909fc9ae66888'), ObjectId
('69e822d1cfc909fc9ae66889'), ObjectId('69e822d1cfc909fc9ae6688a'), ObjectId('69e822d1cfc909fc9ae6688b'),
ObjectId('69e822d1cfc909fc9ae6688c'), ObjectId('69e822d1cfc909fc9ae6688d'), ObjectId('69e822d1cfc909fc9ae6
688e'), ObjectId('69e822d1cfc909fc9ae6688f'), ObjectId('69e822d1cfc909fc9ae66890'), ObjectId('69e822d1cfc9
09fc9ae66891'), ObjectId('69e822d1cfc909fc9ae66892'), ObjectId('69e822d1cfc909fc9ae66893'), ObjectId('69e8
22d1cfc909fc9ae66894'), ObjectId('69e822d1cfc909fc9ae66895'), ObjectId('69e822d1cfc909fc9ae66896'), Object
Id('69e822d1cfc909fc9ae66897'), ObjectId('69e822d1cfc909fc9ae66898'), ObjectId('69e822d1cfc909fc9ae6689
9'), ObjectId('69e822d1cfc909fc9ae6689a'), ObjectId('69e822d1cfc909fc9ae6689b'), ObjectId('69e822d1cfc909f
c9ae6689c'), ObjectId('69e822d1cfc909fc9ae6689d'), ObjectId('69e822d1cfc909fc9ae6689e'), ObjectId('69e822d
1cfc909fc9ae6689f'), ObjectId('69e822d1cfc909fc9ae668a0'), ObjectId('69e822d1cfc909fc9ae668a1'), ObjectId
('69e822d1cfc909fc9ae668a2'), ObjectId('69e822d1cfc909fc9ae668a3'), ObjectId('69e822d1cfc909fc9ae668a4'),
ObjectId('69e822d1cfc909fc9ae668a5'), ObjectId('69e822d1cfc909fc9ae668a6'), ObjectId('69e822d1cfc909fc9ae6
68a7'), ObjectId('69e822d1cfc909fc9ae668a8'), ObjectId('69e822d1cfc909fc9ae668a9'), ObjectId('69e822d1cfc9
09fc9ae668aa'), ObjectId('69e822d1cfc909fc9ae668ab'), ObjectId('69e822d1cfc909fc9ae668ac'), ObjectId('69e8
22d1cfc909fc9ae668ad'), ObjectId('69e822d1cfc909fc9ae668ae'), ObjectId('69e822d1cfc909fc9ae668af'), Object
Id('69e822d1cfc909fc9ae668b0'), ObjectId('69e822d1cfc909fc9ae668b1'), ObjectId('69e822d1cfc909fc9ae668b
2'), ObjectId('69e822d1cfc909fc9ae668b3'), ObjectId('69e822d1cfc909fc9ae668b4'), ObjectId('69e822d1cfc909f
c9ae668b5'), ObjectId('69e822d1cfc909fc9ae668b6'), ObjectId('69e822d1cfc909fc9ae668b7'), ObjectId('69e822d
1cfc909fc9ae668b8'), ObjectId('69e822d1cfc909fc9ae668b9'), ObjectId('69e822d1cfc909fc9ae668ba'), ObjectId
('69e822d1cfc909fc9ae668bb'), ObjectId('69e822d1cfc909fc9ae668bc'), ObjectId('69e822d1cfc909fc9ae668bd'),
ObjectId('69e822d1cfc909fc9ae668be'), ObjectId('69e822d1cfc909fc9ae668bf'), ObjectId('69e822d1cfc909fc9ae6
68c0')], acknowledged=True)
```

In [112…
```
with open("orders.json", "r") as in_file:
    orders = json.load(in_file)
```

In [113…
```
orders[0:2]
```

Out[113… [{'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'orderDetails': [{'productCode': 'S18_1749',
    'quantityOrdered': 30,
    'priceEach': 136.0,
    'orderLineNumber': 3},
   {'productCode': 'S18_2248',
    'quantityOrdered': 50,
    'priceEach': 55.09,
    'orderLineNumber': 2},
   {'productCode': 'S18_4409',
    'quantityOrdered': 22,
    'priceEach': 75.46,
    'orderLineNumber': 4},
   {'productCode': 'S24_3969',
    'quantityOrdered': 49,
    'priceEach': 35.29,
    'orderLineNumber': 1}]},
 {'orderNumber': 10101,
  'orderDate': '2003-01-09',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped',
  'comments': 'Check on availability.',
  'customerNumber': 128,
  'orderDetails': [{'productCode': 'S18_2325',
    'quantityOrdered': 25,
    'priceEach': 108.06,
    'orderLineNumber': 4},
   {'productCode': 'S18_2795',
    'quantityOrdered': 26,
    'priceEach': 167.06,
    'orderLineNumber': 1},
   {'productCode': 'S24_1937',
    'quantityOrdered': 45,
    'priceEach': 32.53,
    'orderLineNumber': 3},
   {'productCode': 'S24_2022',
    'quantityOrdered': 46,
    'priceEach': 44.35,
    'orderLineNumber': 2}]}]

In [114…
```
len(orders)
```

Out[114… 326

In [115…
```
#BE CAREFUL. DO NOT LOAD THIS MULTIPLE TIMES. THE LENGTH SHOULD BE 326 IN MongdDB Cloud Atlas.
client['classicmodels']['orders'].insert_many(orders)
```

Out[115… InsertManyResult([ObjectId('69e822e8cfc909fc9ae668c1'), ObjectId('69e822e8cfc909fc9ae668c2'), ObjectId('69e822e8cfc909fc9ae668c3'), ObjectId('69e822e8cfc909fc9ae668c4'), ObjectId('69e822e8cfc909fc9ae668c5'), ObjectId('69e822e8cfc909fc9ae668c6'), ObjectId('69e822e8cfc909fc9ae668c7'), ObjectId('69e822e8cfc909fc9ae668c8'), ObjectId('69e822e8cfc909fc9ae668c9'), ObjectId('69e822e8cfc909fc9ae668ca'), ObjectId('69e822e8cfc909fc9ae668cb'), ObjectId('69e822e8cfc909fc9ae668cc'), ObjectId('69e822e8cfc909fc9ae668cd'), ObjectId('69e822e8cfc909fc9ae668ce'), ObjectId('69e822e8cfc909fc9ae668cf'), ObjectId('69e822e8cfc909fc9ae668d0'), ObjectId('69e822e8cfc909fc9ae668d1'), ObjectId('69e822e8cfc909fc9ae668d2'), ObjectId('69e822e8cfc909fc9ae668d3'), ObjectId('69e822e8cfc909fc9ae668d4'), ObjectId('69e822e8cfc909fc9ae668d5'), ObjectId('69e822e8cfc909fc9ae668d6'), ObjectId('69e822e8cfc909fc9ae668d7'), ObjectId('69e822e8cfc909fc9ae668d8'), ObjectId('69e822e8cfc909fc9ae668d9'), ObjectId('69e822e8cfc909fc9ae668da'), ObjectId('69e822e8cfc909fc9ae668db'), ObjectId('69e822e8cfc909fc9ae668dc'), ObjectId('69e822e8cfc909fc9ae668dd'), ObjectId('69e822e8cfc909fc9ae668de'), ObjectId('69e822e8cfc909fc9ae668df'), ObjectId('69e822e8cfc909fc9ae668e0'), ObjectId('69e822e8cfc909fc9ae668e1'), ObjectId('69e822e8cfc909fc9ae668e2'), ObjectId('69e822e8cfc909fc9ae668e3'), ObjectId('69e822e8cfc909fc9ae668e4'), ObjectId('69e822e8cfc909fc9ae668e5'), ObjectId('69e822e8cfc909fc9ae668e6'), ObjectId('69e822e8cfc909fc9ae668e7'), ObjectId('69e822e8cfc909fc9ae668e8'), ObjectId('69e822e8cfc909fc9ae668e9'), ObjectId('69e822e8cfc909fc9ae668ea'), ObjectId('69e822e8cfc909fc9ae668eb'), ObjectId('69e822e8cfc909fc9ae668ec'), ObjectId('69e822e8cfc909fc9ae668ed'), ObjectId('69e822e8cfc909fc9ae668ee'), ObjectId('69e822e8cfc909fc9ae668ef'), ObjectId('69e822e8cfc909fc9ae668f0'), ObjectId('69e822e8cfc909fc9ae668f1'), ObjectId('69e822e8cfc909fc9ae668f2'), ObjectId('69e822e8cfc909fc9ae668f3'), ObjectId('69e822e8cfc909fc9ae668f4'), ObjectId('69e822e8cfc909fc9ae668f5'), ObjectId('69e822e8cfc909fc9ae668f6'), ObjectId('69e822e8cfc909fc9ae668f7'), ObjectId('69e822e8cfc909fc9ae668f8'), ObjectId('69e822e8cfc909fc9ae668f9'), ObjectId('69e822e8cfc909fc9ae668fa'), ObjectId('69e822e8cfc909fc9ae668fb'), ObjectId('69e822e8cfc909fc9ae668fc'), ObjectId('69e822e8cfc909fc9ae668fd'), ObjectId('69e822e8cfc909fc9ae668fe'), ObjectId('69e822e8cfc909fc9ae668ff'), ObjectId('69e822e8cfc909fc9ae66900'), ObjectId('69e822e8cfc909fc9ae66901'), ObjectId('69e822e8cfc909fc9ae66902'), ObjectId('69e822e8cfc909fc9ae66903'), ObjectId('69e822e8cfc909fc9ae66904'), ObjectId('69e822e8cfc909fc9ae66905'), ObjectId('69e822e8cfc909fc9ae66906'), ObjectId('69e822e8cfc909fc9ae66907'), ObjectId('69e822e8cfc909fc9ae66908'), ObjectId('69e822e8cfc909fc9ae66909'), ObjectId('69e822e8cfc909fc9ae6690a'), ObjectId('69e822e8cfc909fc9ae6690b'), ObjectId('69e822e8cfc909fc9ae6690c'), ObjectId('69e822e8cfc909fc9ae6690d'), ObjectId('69e822e8cfc909fc9ae6690e'), ObjectId('69e822e8cfc909fc9ae6690f'), ObjectId('69e822e8cfc909fc9ae66910'), ObjectId('69e822e8cfc909fc9ae66911'), ObjectId('69e822e8cfc909fc9ae66912'), ObjectId('69e822e8cfc909fc9ae66913'), ObjectId('69e822e8cfc909fc9ae66914'), ObjectId('69e822e8cfc909fc9ae66915'), ObjectId('69e822e8cfc909fc9ae66916'), ObjectId('69e822e8cfc909fc9ae66917'), ObjectId('69e822e8cfc909fc9ae66918'), ObjectId('69e822e8cfc909fc9ae66919'), ObjectId('69e822e8cfc909fc9ae6691a'), ObjectId('69e822e8cfc909fc9ae6691b'), ObjectId('69e822e8cfc909fc9ae6691c'), ObjectId('69e822e8cfc909fc9ae6691d'), ObjectId('69e822e8cfc909fc9ae6691e'), ObjectId('69e822e8cfc909fc9ae6691f'), ObjectId('69e822e8cfc909fc9ae66920'), ObjectId('69e822e8cfc909fc9ae66921'), ObjectId('69e822e8cfc909fc9ae66922'), ObjectId('69e822e8cfc909fc9ae66923'), ObjectId('69e822e8cfc909fc9ae66924'), ObjectId('69e822e8cfc909fc9ae66925'), ObjectId('69e822e8cfc909fc9ae66926'), ObjectId('69e822e8cfc909fc9ae66927'), ObjectId('69e822e8cfc909fc9ae66928'), ObjectId('69e822e8cfc909fc9ae66929'), ObjectId('69e822e8cfc909fc9ae6692a'), ObjectId('69e822e8cfc909fc9ae6692b'), ObjectId('69e822e8cfc909fc9ae6692c'), ObjectId('69e822e8cfc909fc9ae6692d'), ObjectId('69e822e8cfc909fc9ae6692e'), ObjectId('69e822e8cfc909fc9ae6692f'), ObjectId('69e822e8cfc909fc9ae66930'), ObjectId('69e822e8cfc909fc9ae66931'), ObjectId('69e822e8cfc909fc9ae66932'), ObjectId('69e822e8cfc909fc9ae66933'), ObjectId('69e822e8cfc909fc9ae66934'), ObjectId('69e822e8cfc909fc9ae66935'), ObjectId('69e822e8cfc909fc9ae66936'), ObjectId('69e822e8cfc909fc9ae66937'), ObjectId('69e822e8cfc909fc9ae66938'), ObjectId('69e822e8cfc909fc9ae66939'), ObjectId('69e822e8cfc909fc9ae6693a'), ObjectId('69e822e8cfc909fc9ae6693b'), ObjectId('69e822e8cfc909fc9ae6693c'), ObjectId('69e822e8cfc909fc9ae6693d'), ObjectId('69e822e8cfc909fc9ae6693e'), ObjectId('69e822e8cfc909fc9ae6693f'), ObjectId('69e822e8cfc909fc9ae66940'), ObjectId('69e822e8cfc909fc9ae66941'), ObjectId('69e822e8cfc909fc9ae66942'), ObjectId('69e822e8cfc909fc9ae66943'), ObjectId('69e822e8cfc909fc9ae66944'), ObjectId('69e822e8cfc909fc9ae66945'), ObjectId('69e822e8cfc909fc9ae66946'), ObjectId('69e822e8cfc909fc9ae66947'), ObjectId('69e822e8cfc909fc9ae66948'), ObjectId('69e822e8cfc909fc9ae66949'), ObjectId('69e822e8cfc909fc9ae6694a'), ObjectId('69e822e8cfc909fc9ae6694b'), ObjectId('69e822e8cfc909fc9ae6694c'), ObjectId('69e822e8cfc909fc9ae6694d'), ObjectId('69e822e8cfc909fc9ae6694e'), ObjectId('69e822e8cfc909fc9ae6694f'), ObjectId('69e822e8cfc909fc9ae66950'), ObjectId('69e822e8cfc909fc9ae66951'), ObjectId('69e822e8cfc909fc9ae66952'), ObjectId('69e822e8cfc909fc9ae66953'), ObjectId('69e822e8cfc909fc9ae66954'), ObjectId('69e822e8cfc909fc9ae66955'), ObjectId('69e822e8cfc909fc9ae66956'), ObjectId('69e822e8cfc909fc9ae66957'), ObjectId('69e822e8cfc909fc9ae66958'), ObjectId('69e822e8cfc909fc9ae66959'), ObjectId('69e822e8cfc909fc9ae6695a'), ObjectId('69e822e8cfc909fc9ae6695b'), ObjectId('69e822e8cfc909fc9ae6695c'), ObjectId('69e822e8cfc909fc9ae6695d'), ObjectId('69e822e8cfc909fc9ae6695e'), ObjectId('69e822e8cfc909fc9ae6695f'), ObjectId('69e822e8cfc909fc9ae66960'), ObjectId('69e822e8cfc909fc9ae66961'), ObjectId('69e822e8cfc909fc9ae66962'), ObjectId('69e822e8cfc909fc9ae66963'), ObjectId('69e822e8cfc909fc9ae66964'), ObjectId('69e822e8cfc909fc9ae66965'), ObjectId('69e822e8cfc909fc9ae66966'), ObjectId('69e822e8cfc909fc9ae66967'), ObjectId('69e822e8cfc909fc9ae66968'), ObjectId('69e822e8cfc909fc9ae66969'), ObjectId('69e822e8cfc909fc9ae6696a'), ObjectId('69e822e8cfc909fc9ae6696b'), ObjectId('69e822e8cfc909fc9ae6696c'), ObjectId('69e822e8cfc909fc9ae6696d'), ObjectId('69e822e8cfc909fc9ae6696e'), ObjectId('69e822e8cfc909fc9ae6696f'), ObjectId('69e822e8cfc909fc9ae66970'), ObjectId('69e822e8cfc909fc9ae66971'), ObjectId('69e822e8cfc909fc9ae66972'), ObjectId('69e822e8cfc909fc9ae66973'), ObjectId('69e822e8cfc909fc9ae66974'), ObjectId('69e822e8cfc909fc9ae66975'), ObjectId('69e822e8cfc909fc9ae66976'), ObjectId('69e822e8cfc909fc9ae66977'), ObjectId('69e822e8cfc909fc9ae66978'), ObjectId('69e822e8cfc909fc9ae66979'), ObjectId('69e822e8cfc909fc9ae6697a'), ObjectId('69e822e8cfc909fc9ae6697b'), ObjectId('69e822e8cfc909fc9ae6697c'), ObjectId('69e822e8cfc909fc9ae6697d'), ObjectId('69e822e8cfc909fc9ae6697e'), ObjectId('69e822e8cfc909fc9ae6697f'), ObjectId('69e822e8cfc909fc9ae66980'), ObjectId('69e822e8cfc909fc9ae66981'), ObjectId('69e822e8cfc909fc9ae66982'), ObjectId('69e822e8cfc909fc9ae66983'), ObjectId('69e822e8cfc909fc9ae66984'), ObjectId('69e822e8cfc909fc9ae66985'), ObjectId('69e822e8cfc909fc9ae66986'), ObjectId('69e822e8cfc909fc9ae66987'), ObjectId('69e822e8cfc909fc9ae66988'), ObjectId('69e822e8cfc909fc9ae66989'), ObjectId('69e822e8cfc909fc9ae6698a'), ObjectId('69e822e8cfc909fc9ae6698b'), ObjectId('69e822e8cfc909fc9ae6698c'), ObjectId('69e822e8cfc909fc9ae6698d'), ObjectId('69e822e8cfc909fc9ae6698e'), ObjectId('69e822e8cfc909fc9ae6698f'), ObjectId('69e822e8cfc909fc9ae66990'), ObjectId('69e822e8cfc909fc9ae66991'), ObjectId('69e822e8cfc909fc9ae66992'), ObjectId('69e822e8cfc909fc9ae66993'), ObjectId('69e822e8cfc909fc9ae66994'), ObjectId('69e822e8cfc909fc9ae66995'), ObjectId('69e822e8cfc909fc9ae66996'), ObjectId('69e822e8cfc909fc9ae66997'), ObjectId('69e822e8cfc909fc9ae66998'), ObjectId('69e822e8cfc909fc9ae66999'), ObjectId('69e822e8cfc909fc9ae6699a'), ObjectId('69e822e8cfc909fc9ae6699b'), ObjectId('69e822e8cfc909fc9ae6699c'), ObjectId('69e822e8cfc909fc9ae6699d'), ObjectId('69e822e8cfc909fc9ae6699e'), ObjectId('69e822e8cfc909fc9ae6699f'), ObjectId('69e822e8cfc909fc9ae669a0'), ObjectId('69e822e8cfc909fc9ae669a1'), ObjectId('69e822e8cfc909fc9ae669a2'), ObjectId('69e822e8cfc909fc9ae669a3'), ObjectId('69e822e8cfc909fc9ae669a4'), ObjectId('69e822e8cfc909fc9ae669a5'), ObjectId('69e822e8cfc909fc9ae669a6'), ObjectId('69e822e8cfc909fc9ae669a7'), ObjectId('69e822e8cfc909fc9ae669a8'), ObjectId('69e822e8cfc909fc9ae669a9'), ObjectId('69e822e8cfc909fc9ae669aa'), ObjectId('69e822e8cfc909fc9ae669ab'), ObjectId('69e822e8cfc909fc9ae669ac'), ObjectId('69e822e8cfc909fc9ae669ad'), ObjectId('69e822e8cfc909fc9ae669ae'), ObjectId('69e822e8cfc909fc9ae669af'), ObjectId('69e822e8cfc909fc9ae669b0'), ObjectId('69e822e8cfc909fc9ae669b1'), ObjectId('69e822e8cfc909fc9ae669b2'), ObjectId('69e822e8cfc909fc9ae669b3'), ObjectId('69e822e8cfc909fc9ae669b4'), ObjectId('69e822e8cfc909fc9ae669b5'), ObjectId('69e822e8cfc909fc9ae669b6'), ObjectId('69e822e8cfc909fc9ae669b7'), ObjectId('69e822e8cfc909fc9ae669b8'), ObjectId('69e822e8cfc909fc9ae669b9'), ObjectId('69e822e8cfc9

```
09fc9ae669ba'), ObjectId('69e822e8cfc909fc9ae669bb'), ObjectId('69e822e8cfc909fc9ae669bc'), ObjectId('69e8
22e8cfc909fc9ae669bd'), ObjectId('69e822e8cfc909fc9ae669be'), ObjectId('69e822e8cfc909fc9ae669bf'), Object
Id('69e822e8cfc909fc9ae669c0'), ObjectId('69e822e8cfc909fc9ae669c1'), ObjectId('69e822e8cfc909fc9ae669c
2'), ObjectId('69e822e8cfc909fc9ae669c3'), ObjectId('69e822e8cfc909fc9ae669c4'), ObjectId('69e822e8cfc909f
c9ae669c5'), ObjectId('69e822e8cfc909fc9ae669c6'), ObjectId('69e822e8cfc909fc9ae669c7'), ObjectId('69e822e
8cfc909fc9ae669c8'), ObjectId('69e822e8cfc909fc9ae669c9'), ObjectId('69e822e8cfc909fc9ae669ca'), ObjectId
('69e822e8cfc909fc9ae669cb'), ObjectId('69e822e8cfc909fc9ae669cc'), ObjectId('69e822e8cfc909fc9ae669cd'),
ObjectId('69e822e8cfc909fc9ae669ce'), ObjectId('69e822e8cfc909fc9ae669cf'), ObjectId('69e822e8cfc909fc9ae6
69d0'), ObjectId('69e822e8cfc909fc9ae669d1'), ObjectId('69e822e8cfc909fc9ae669d2'), ObjectId('69e822e8cfc9
09fc9ae669d3'), ObjectId('69e822e8cfc909fc9ae669d4'), ObjectId('69e822e8cfc909fc9ae669d5'), ObjectId('69e8
22e8cfc909fc9ae669d6'), ObjectId('69e822e8cfc909fc9ae669d7'), ObjectId('69e822e8cfc909fc9ae669d8'), Object
Id('69e822e8cfc909fc9ae669d9'), ObjectId('69e822e8cfc909fc9ae669da'), ObjectId('69e822e8cfc909fc9ae669d
b'), ObjectId('69e822e8cfc909fc9ae669dc'), ObjectId('69e822e8cfc909fc9ae669dd'), ObjectId('69e822e8cfc909f
c9ae669de'), ObjectId('69e822e8cfc909fc9ae669df'), ObjectId('69e822e8cfc909fc9ae669e0'), ObjectId('69e822e
8cfc909fc9ae669e1'), ObjectId('69e822e8cfc909fc9ae669e2'), ObjectId('69e822e8cfc909fc9ae669e3'), ObjectId
('69e822e8cfc909fc9ae669e4'), ObjectId('69e822e8cfc909fc9ae669e5'), ObjectId('69e822e8cfc909fc9ae669e6'),
ObjectId('69e822e8cfc909fc9ae669e7'), ObjectId('69e822e8cfc909fc9ae669e8'), ObjectId('69e822e8cfc909fc9ae6
69e9'), ObjectId('69e822e8cfc909fc9ae669ea'), ObjectId('69e822e8cfc909fc9ae669eb'), ObjectId('69e822e8cfc9
09fc9ae669ec'), ObjectId('69e822e8cfc909fc9ae669ed'), ObjectId('69e822e8cfc909fc9ae669ee'), ObjectId('69e8
22e8cfc909fc9ae669ef'), ObjectId('69e822e8cfc909fc9ae669f0'), ObjectId('69e822e8cfc909fc9ae669f1'), Object
Id('69e822e8cfc909fc9ae669f2'), ObjectId('69e822e8cfc909fc9ae669f3'), ObjectId('69e822e8cfc909fc9ae669f
4'), ObjectId('69e822e8cfc909fc9ae669f5'), ObjectId('69e822e8cfc909fc9ae669f6'), ObjectId('69e822e8cfc909f
c9ae669f7'), ObjectId('69e822e8cfc909fc9ae669f8'), ObjectId('69e822e8cfc909fc9ae669f9'), ObjectId('69e822e
8cfc909fc9ae669fa'), ObjectId('69e822e8cfc909fc9ae669fb'), ObjectId('69e822e8cfc909fc9ae669fc'), ObjectId
('69e822e8cfc909fc9ae669fd'), ObjectId('69e822e8cfc909fc9ae669fe'), ObjectId('69e822e8cfc909fc9ae669ff'),
ObjectId('69e822e8cfc909fc9ae66a00'), ObjectId('69e822e8cfc909fc9ae66a01'), ObjectId('69e822e8cfc909fc9ae6
6a02'), ObjectId('69e822e8cfc909fc9ae66a03'), ObjectId('69e822e8cfc909fc9ae66a04'), ObjectId('69e822e8cfc9
09fc9ae66a05'), ObjectId('69e822e8cfc909fc9ae66a06')], acknowledged=True)
```

Neo4j

In [33]:
```
import neo4j
```

In [34]:
```
#
# You must change to the information you downloaded when you created your database.
#
NEO4J_URI="Neo4j+ssc://b57b835c.databases.neo4j.io"
NEO4J_USERNAME="b57b835c"
NEO4J_PASSWORD="QSYDWfOpkFexJINS7AdbO3Zx0oqXSw8nH0suS0obicY"
NEO4J_DATABASE="b57b835c"
AURA_INSTANCEID="b57b835c"
AURA_INSTANCENAME="Free instance"
```

In [35]:
```
from neo4j import GraphDatabase

# URI examples: "neo4j://localhost", "neo4j+s://xxx.databases.neo4j.io"

URL = NEO4J_URI

AUTH = (NEO4J_USERNAME, NEO4J_PASSWORD)

def run_query(q):


    with GraphDatabase.driver(NEO4J_URI, auth=AUTH) as driver:
        driver.verify_connectivity()

        records, summary, keys = driver.execute_query(
            q,
            database_=NEO4J_DATABASE,
        )

    # Loop through results and do something with them
    print("\nResult is")
    for record in records:
        print(record.data())  # obtain record as dict
```

In [36]:
```
run_query("MATCH (p:Person)-[:FOLLOWS]->(:Person) RETURN p.name AS name""")
```

```
Result is
{'name': 'Paul Blythe'}
{'name': 'Angela Scope'}
{'name': 'James Thompson'}
```

Written Questions

W1

Question

For relationships in the ER Model, briefly explain the concepts of *degree* and *cardinality*.

Answer

Degree is the number of entity sets that participate in a relationship.

- Unary (Degree 1): An entity relates to itself (e.g., an Employee manages another Employee).
- Binary (Degree 2): Two entities are involved (e.g., Customer buys Product).
- Ternary (Degree 3): Three entities are involved (e.g., Vendor supplies Parts to a Project).

Cardinality is the number of entities to which another entity can be associated via a relationship set. For a binary relationship set the mapping cardinality must be one of the following types:

- One to one
- One to many
- Many to one
- Many to many

W2

Question

Briefly explain the similarities and differences between *functions, procedures* and *triggers*.

Answer

All three are server-side routines stored in the database, but they differ along three dimensions — whether they can change data, whether they return a value, and how they are invoked.

- Functions are read-only in spirit: they do not change data and they always return a value. Because they return a value, they are invoked inside a SQL statement (e.g., SELECT my_fn(x) FROM … or in a WHERE clause), where the returned value is used as part of the expression.
- Procedures are the general-purpose routines: they can change data (INSERT/UPDATE/DELETE, transaction control, etc.) and they sometimes return a value — only when the developer chooses to, typically via OUT / INOUT parameters or by producing a result set. Because they are not expressions, they are invoked explicitly by the caller (EXEC / CALL proc(...)), not embedded inside a query.
- Triggers also can change data, but never return a value to a caller. That's because they are not called at all — they run as a reaction to a data event (BEFORE/AFTER an INSERT, UPDATE, or DELETE on a specified table), with the DBMS firing them automatically and passing them the affected OLD/NEW rows.

W3

Question

Consider the table below and using your knowledge of the the db_book sample DB associated with the sample textbook, identify three functional dependencies in the table below.

	ID	name	dept_name	salary	building	budget
1	76766	Crick	Biology	72000.00	Watson	90000.00
2	10101	Srinivasan	Comp. Sci.	65000.00	Taylor	100000.00
3	45565	Katz	Comp. Sci.	75000.00	Taylor	100000.00
4	83821	Brandt	Comp. Sci.	92000.00	Taylor	100000.00
5	98345	Kim	Elec. Eng.	80000.00	Taylor	85000.00
6	12121	Wu	Finance	90000.00	Painter	120000.00
7	76543	Singh	Finance	80000.00	Painter	120000.00
8	32343	El Said	History	60000.00	Painter	50000.00
9	58583	Califieri	History	62000.00	Painter	50000.00
10	15151	Mozart	Music	40000.00	Packard	80000.00
11	22222	Einstein	Physics	95000.00	Watson	70000.00
12	33456	Gold	Physics	87000.00	Watson	70000.00

Answer

Functional dependency require that the value for a certain set of attributes determines uniquely the value for another set of attributes. A functional dependency is a generalization of the notion of a key.

- ID -> name (Instructors are uniquely identified by their ID.)
- dept_name → budget (Each department has exactly one budget — also from department.)
- dept_name → building (Each department is located in exactly one building — this comes from department in db_book.)

W4

Question

Briefly compare BCNF and 3NF.

Answer

Both are normal forms that aim to eliminate update/insert/delete anomalies caused by functional dependencies.

BCNF is stricter than 3NF. In BCNF, every nontrivial functional dependency must have a determinant that is a superkey. 3NF relaxes that rule: a dependency is still allowed if the dependent attribute is part of some candidate key. So BCNF removes more redundancy and update anomalies, but it may fail to preserve all dependencies. 3NF is weaker, but it is easier to get a lossless, dependency-preserving design.

W5

Question

What are some benefits of RAID-5 compared to RAID-0 and RAID-1?

Answer

RAID-0 (striping, no redundancy): fast reads/writes and 100% capacity utilization, but no fault tolerance — a single disk failure loses all data.

RAID-1 (mirroring): good read performance and simple fault tolerance, but only 50% of raw capacity is usable (every block is stored twice).

RAID-5 (block-level striping with distributed parity): gives you the best of both worlds relative to those two:

- Fault tolerance — can survive the failure of any one disk (rebuild from parity), unlike RAID-0.
- Higher usable capacity / better storage efficiency — for N disks, usable capacity is (N–1)/N, versus only 1/2 for RAID-1.
- Good read throughput — reads are striped across all disks, similar to RAID-0 and better than RAID-1.
- Parity is distributed across all disks (unlike RAID-4), so there is no single parity-disk bottleneck

W6

Question

For what types of query might columnar file organization be better than row oriented organization.

Answer

Columnar organization is better for queries that scan many rows but only a few columns, especially SUM, AVG, COUNT, GROUP BY, filtering, and reporting queries. It reduces I/O because the system reads only the needed columns, and it often compresses better because values in a column are similar.

Row-oriented is better when workloads are fetching or updating entire rows by primary key (e.g., SELECT * FROM customer WHERE id = __, or point inserts/updates), where the whole row is read/written together.

W7

Question

For what types of query is LRU replacement a very bad buffer replacement policy?

Answer

LRU (Least Recently Used) assumes recently used pages are likely to be used again soon. It performs poorly when that assumption is violated. It is especially bad for large sequential scans and nested-loop join workloads. In those cases, pages may be read once and not reused soon, but LRU still keeps the most recently scanned pages in memory.

So policies like MRU (Most Recently Used) can work better for scan-heavy workloads.

S8

Question

How many clustered indexes can a table have? Why?

Answer

A table can have only one clustered index. The reason is that a clustered index determines the sequential order of the data records in the file, and the table can only be sequentially ordered one way at a time. You can still have multiple nonclustered/secondary indexes.

- Clustering index: in a sequentially ordered file, the index whose search key specifies the sequential order of the file. Also called primary index.
- Secondary index: an index whose search key specifies an order different from the sequential order of the file. Also called nonclustering index.

S9

Question

Briefly explain the concept of *index selectivity* and how it relates to query processing/optimization?

Answer

Selectivity measures how well an index narrows down the rows returned by a predicate. A highly selective index returns a small fraction of rows (close to 0), a low-selectivity one returns a large fraction.

This matters because the optimizer usually prefers more selective indexes first, since they reduce the number of tuples and blocks that must be read and processed. If selectivity is poor, a full scan may be cheaper than using the index.

S10

Question

Consider the following query for the db_book sample database associated with the recommended textbook.

```
select
    *
from
    student join takes using(ID)
where
    student.dept_name = 'Comp. Sci.';
```

What is an equivalent query that might be more efficient?

Answer

SELECT *

FROM (SELECT * FROM student WHERE dept_name = 'Comp. Sci.') AS s

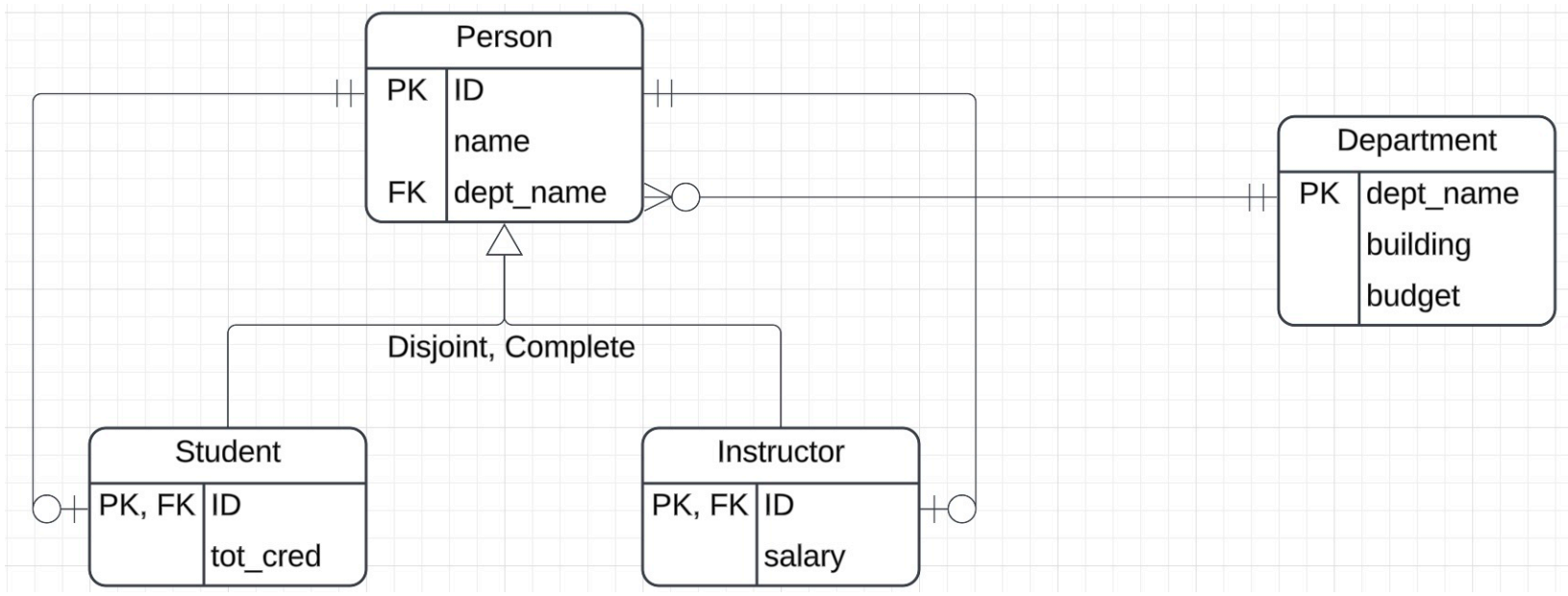
JOIN takes USING (ID);

Applying dept_name='Comp. Sci.' on student before the join dramatically reduces the size of the left input to the join (only CS students, not the whole student table).

SQL

S1

Consider the modification to the db_book schema depicted below. Write DDL create table and create view statements to implement the inheritance/specialization in SQL. Execute your statements in the cell below.



In [148…

```sql
%%sql

drop schema if exists hw3;
create schema hw3;
use hw3;

-- ------------------------------------------------------------
-- Department
-- ------------------------------------------------------------
create table department (
    dept_name varchar(64)    not null,
    building  varchar(64),
    budget    decimal(12,2),
    constraint pk_department primary key (dept_name)
);

-- ------------------------------------------------------------
-- Person: ONE wide table holding the superclass AND both
-- subclasses' local attributes.
--
--   person_type is the discriminator for the Disjoint +
--   Complete specialization.
--
--   Subclass-local columns (tot_cred, salary) are nullable at
--   the column level, but CHECK constraints enforce:
--     - Student rows:    tot_cred NOT NULL, salary IS NULL
--     - Instructor rows: salary   NOT NULL, tot_cred IS NULL
--   This gives us disjointness without triggers.
-- ------------------------------------------------------------
create table person (
    ID          int unsigned  not null,
    name        varchar(64)  not null,
    dept_name   varchar(64),
    person_type enum('Student','Instructor') not null,

    -- Student-only attribute
    tot_cred    decimal(6,2),
    -- Instructor-only attribute
    salary      decimal(10,2),

    constraint pk_person primary key (ID),

    constraint fk_person_dept
        foreign key (dept_name) references department(dept_name)
        on delete set null on update cascade,

    -- A Student row must have tot_cred and no salary.
    constraint chk_student_cols
        check (person_type <> 'Student'
               or (tot_cred is not null and salary is null)),

    -- An Instructor row must have salary and no tot_cred.
    constraint chk_instructor_cols
        check (person_type <> 'Instructor'
               or (salary is not null and tot_cred is null))
);

-- ------------------------------------------------------------
-- Views: expose each subclass as if it were its own table,
-- hiding the columns that don't belong to that subclass.
-- ------------------------------------------------------------
create or replace view student as
select ID, name, dept_name, tot_cred
from   person
where  person_type = 'Student';

create or replace view instructor as
select ID, name, dept_name, salary
from   person
where  person_type = 'Instructor';
```

```
 * mysql+pymysql://root:***@localhost
4 rows affected.
1 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
```

Out[148… []

S2

Given your design above, write a function that computes a unique `ID` for a newly created `Person.` Execute your function and show the results.

In [150…

```
%%sql

use hw3;

drop function if exists next_person_id;

-- Unique-ID generator for a newly created Person.
-- Strategy (per professor guidance): MAX(ID) + 1.
-- Reads from `person`, so its result depends on table state
-- => NOT DETERMINISTIC, READS SQL DATA.
create function next_person_id()
    returns int unsigned
    reads sql data
    not deterministic
    return (select coalesce(max(ID), 0) + 1 from person);


select next_person_id() as new_person_id;
```

```
 * mysql+pymysql://root:***@localhost
0 rows affected.
0 rows affected.
0 rows affected.
1 rows affected.
```

Out[150…

new_person_id
1

S3

Consider the following table computed from the tables `student, takes` and `course` in db_book

ID	name	tot_cred	computed_credits	courses	grades	course_credits
00128	Zhang	102	7	CS-101,CS-347	A,A-	4,3
12345	Shankar	32	14	CS-101,CS-190,CS-315,CS-347	C,A,A,A	4,4,3,3
19991	Brandt	80	3	HIS-351	B	3
23121	Chavez	110	3	FIN-201	C+	3
44553	Peltier	56	4	PHY-101	B-	4
45678	Levy	46	7	CS-101,CS-101,CS-319	F,B+,B	0,4,3
54321	Williams	54	8	CS-101,CS-190	A-,B+	4,4
55739	Sanchez	38	3	MU-199	A-	3
70557	Snow	0	0	<null>	<null>	<null>
76543	Brown	58	7	CS-101,CS-319	A,A	4,3
76653	Aoi	60	3	EE-181	C	3
98765	Bourikas	98	7	CS-101,CS-315	C-,B	4,3
98988	Tanaka	120	4	BIO-101,BIO-301	A,I	4,0

The columns are computed using the following rules:

- `ID, name` and `tot_cred` come from `student`.
- `courses` is a list of the `course_ids` for the courses a student took. The list is `null` if the student did not take any courses.
- `grades` is a list of grades the student earned in the courses and is computed from the following rules:
  - If the student did not take any course, the value is `null.`
  - If the student took a course but the grade was `null`, the value is `I`
  - Otherwise the value is the student's grade in the course.
- `course_credits` is a list of the `course.credits` for the courses a student took.
  - The list is `null` if the student did not take any courses.
  - The student earned a `0` in the course if the grade is `F` or `I`.
  - Otherwise the list contains the the credits for the course.

Write and execute SQL statement that produces the table.

In [154…

```
%%sql

use db_book;
```

```sql
SELECT s.ID,
    s.name,
    s.tot_cred,
    COALESCE(
        SUM(
            CASE
                WHEN t.course_id IS NULL THEN 0
                WHEN COALESCE(t.grade, 'I') IN ('F', 'I') THEN 0
                ELSE c.credits
            END
        ),
        0
    ) AS computed_credits,
    GROUP_CONCAT(
        CASE
            WHEN t.course_id IS NULL THEN NULL
            ELSE t.course_id
        END
        ORDER BY t.course_id, t.year, FIELD(t.semester, 'Spring', 'Summer', 'Fall', 'Winter')
        SEPARATOR ','
    ) AS courses,
    GROUP_CONCAT(
        CASE
            WHEN t.course_id IS NULL THEN NULL
            ELSE COALESCE(t.grade, 'I')
        END
        ORDER BY t.course_id, t.year, FIELD(t.semester, 'Spring', 'Summer', 'Fall', 'Winter')
        SEPARATOR ','
    ) AS grades,
    GROUP_CONCAT(
        CASE
            WHEN t.course_id IS NULL THEN NULL
            WHEN COALESCE(t.grade, 'I') IN ('F', 'I') THEN 0
            ELSE c.credits
        END
        ORDER BY t.course_id, t.year, FIELD(t.semester, 'Spring', 'Summer', 'Fall', 'Winter')
        SEPARATOR ','
    ) AS course_credits
FROM student s
LEFT JOIN takes t
    ON s.ID = t.ID
LEFT JOIN course c
    ON t.course_id = c.course_id
GROUP BY s.ID, s.name, s.tot_cred
ORDER BY s.ID;
```

```
 * mysql+pymysql://root:***@localhost
0 rows affected.
13 rows affected.
```

Out[154…

ID	name	tot_cred	computed_credits	courses	grades	course_credits
00128	Zhang	102	7	CS-101,CS-347	A,A-	4,3
12345	Shankar	32	14	CS-101,CS-190,CS-315,CS-347	C,A,A,A	4,4,3,3
19991	Brandt	80	3	HIS-351	B	3
23121	Chavez	110	3	FIN-201	C+	3
44553	Peltier	56	4	PHY-101	B-	4
45678	Levy	46	7	CS-101,CS-101,CS-319	F,B+,B	0,4,3
54321	Williams	54	8	CS-101,CS-190	A-,B+	4,4
55739	Sanchez	38	3	MU-199	A-	3
70557	Snow	0	0	None	None	None
76543	Brown	58	7	CS-101,CS-319	A,A	4,3
76653	Aoi	60	3	EE-181	C	3
98765	Bourikas	98	7	CS-101,CS-315	C-,B	4,3
98988	Tanaka	120	4	BIO-101,BIO-301	A,I	4,0

MongoDB

Use the database and collections you created in the setup for these questions.

M1

Produce a list of `customers` in "France." Your answer should only contain the fields `customerNumber, customerName, address`.

In [128…

```
#
# Set your filter expression and projection expression below.
#
```

```
filter = {"address.country": "France"}
projection = {"_id": 0, "customerNumber": 1, "customerName": 1, "address": 1}
```

In [129…
```
#
# Execute this cell to perform the query.
#
result = client["classicmodels"]["customers"].find(
    filter = filter,
    projection = projection
)
result = list(result)
```

In [130…
```
#
# Execute this cell to validate your result
#
len(result)
```

Out[130… 12

In [131…
```
#
# Execute this cell to display and validate your results.
#
result[0:2]
```

Out[131…
```
[{'customerNumber': 103,
  'customerName': 'Atelier graphique',
  'address': {'addressLine1': '54, rue Royale',
   'addressLine2': None,
   'city': 'Nantes',
   'state': None,
   'country': 'France',
   'postalCode': '44000'}},
 {'customerNumber': 119,
  'customerName': 'La Rochelle Gifts',
  'address': {'addressLine1': '67, rue des Cinquante Otages',
   'addressLine2': None,
   'city': 'Nantes',
   'state': None,
   'country': 'France',
   'postalCode': '44000'}}]
```

M2

Write a pipeline that produces a list of documents of the form:

- `orderNumber`
- `orderDate`
- `status`
- `requiredDate`
- `comments`
- `shippedDate`
- `customerNumber`
- `productCode`
- `quantityOrdered`
- `priceEach`
- `orderLineNumber`

Your list of documents should be sorted by `orderNumber` and `orderLineNumber`.

In [139…
```
pipeline = [
    {"$unwind": "$orderDetails"},
    {
        "$project": {
            "_id": 0,
            "orderNumber": 1,
            "orderDate": 1,
            "status": 1,
            "requiredDate": 1,
            "comments": 1,
            "shippedDate": 1,
            "customerNumber": 1,
            "productCode": "$orderDetails.productCode",
            "quantityOrdered": "$orderDetails.quantityOrdered",
            "priceEach": "$orderDetails.priceEach",
            "orderLineNumber": "$orderDetails.orderLineNumber",
        }
    },
    {"$sort": {"orderNumber": 1, "orderLineNumber": 1}},
]
```

In [140…
```
#
# Execute this cell to run your query.
#
result = client['classicmodels']['orders'].aggregate(
```

```
    pipeline
)
result = list(result)
```

In [141…

```
#
# Execute this cell to validate your query.
#
len(result)
```

Out[141… 2996

In [142…

```
#
# Execute this cell to validate and display your results.
#
result[0:20]
```

Out[142…


```
[{'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S24_3969',
  'quantityOrdered': 49,
  'priceEach': 35.29,
  'orderLineNumber': 1},
 {'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S18_2248',
  'quantityOrdered': 50,
  'priceEach': 55.09,
  'orderLineNumber': 2},
 {'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S18_1749',
  'quantityOrdered': 30,
  'priceEach': 136.0,
  'orderLineNumber': 3},
 {'orderNumber': 10100,
  'orderDate': '2003-01-06',
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped',
  'comments': None,
  'customerNumber': 363,
  'productCode': 'S18_4409',
  'quantityOrdered': 22,
  'priceEach': 75.46,
  'orderLineNumber': 4},
 {'orderNumber': 10101,
  'orderDate': '2003-01-09',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped',
  'comments': 'Check on availability.',
  'customerNumber': 128,
  'productCode': 'S18_2795',
  'quantityOrdered': 26,
  'priceEach': 167.06,
  'orderLineNumber': 1},
 {'orderNumber': 10101,
  'orderDate': '2003-01-09',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped',
  'comments': 'Check on availability.',
  'customerNumber': 128,
  'productCode': 'S24_2022',
  'quantityOrdered': 46,
  'priceEach': 44.35,
  'orderLineNumber': 2},
 {'orderNumber': 10101,
  'orderDate': '2003-01-09',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped',
  'comments': 'Check on availability.',
  'customerNumber': 128,
  'productCode': 'S24_1937',
  'quantityOrdered': 45,
  'priceEach': 32.53,
  'orderLineNumber': 3},
 {'orderNumber': 10101,
  'orderDate': '2003-01-09',
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped',
  'comments': 'Check on availability.',
  'customerNumber': 128,
  'productCode': 'S18_2325',
  'quantityOrdered': 25,
  'priceEach': 108.06,
  'orderLineNumber': 4},
 {'orderNumber': 10102,
  'orderDate': '2003-01-10',
```

```
 'requiredDate': '2003-01-18',
 'shippedDate': '2003-01-14',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 181,
 'productCode': 'S18_1367',
 'quantityOrdered': 41,
 'priceEach': 43.13,
 'orderLineNumber': 1},
{'orderNumber': 10102,
 'orderDate': '2003-01-10',
 'requiredDate': '2003-01-18',
 'shippedDate': '2003-01-14',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 181,
 'productCode': 'S18_1342',
 'quantityOrdered': 39,
 'priceEach': 95.55,
 'orderLineNumber': 2},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S24_2300',
 'quantityOrdered': 36,
 'priceEach': 107.34,
 'orderLineNumber': 1},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S18_2432',
 'quantityOrdered': 22,
 'priceEach': 58.34,
 'orderLineNumber': 2},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S32_1268',
 'quantityOrdered': 31,
 'priceEach': 92.46,
 'orderLineNumber': 3},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S10_4962',
 'quantityOrdered': 42,
 'priceEach': 119.67,
 'orderLineNumber': 4},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S18_4600',
 'quantityOrdered': 36,
 'priceEach': 98.07,
 'orderLineNumber': 5},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S700_2824',
 'quantityOrdered': 42,
 'priceEach': 94.07,
 'orderLineNumber': 6},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
```

```
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S32_3522',
 'quantityOrdered': 45,
 'priceEach': 63.35,
 'orderLineNumber': 7},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S12_1666',
 'quantityOrdered': 27,
 'priceEach': 121.64,
 'orderLineNumber': 8},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S18_4668',
 'quantityOrdered': 41,
 'priceEach': 40.75,
 'orderLineNumber': 9},
{'orderNumber': 10103,
 'orderDate': '2003-01-29',
 'requiredDate': '2003-02-07',
 'shippedDate': '2003-02-02',
 'status': 'Shipped',
 'comments': None,
 'customerNumber': 121,
 'productCode': 'S18_1097',
 'quantityOrdered': 35,
 'priceEach': 94.5,
 'orderLineNumber': 10}]
```

M3

The value of a line in `orderDetails` is `priceEach*quantityOrdered`.

The `totalValue` of an `order` is the sum over all line items of the value of the line.

Compute a list of documents of the form:

- `orderNumber`
- `totalValue`

Your answered should be sorted by `orderNumber.`

In [143…

```
#
# Enter your pipeline here.
#
pipeline = [
    {"$unwind": "$orderDetails"},
    {
        "$group": {
            "_id": "$orderNumber",
            "totalValue": {
                "$sum": {
                    "$multiply": [
                        "$orderDetails.priceEach",
                        "$orderDetails.quantityOrdered",
                    ]
                }
            },
        }
    },
    {
        "$project": {
            "_id": 0,
            "orderNumber": "$_id",
            "totalValue": {"$round": ["$totalValue", 2]},
        }
    },
    {"$sort": {"orderNumber": 1}},
]
```

In [144…

```
#
# Execute this cell to run your query.
#
result = client['classicmodels']['orders'].aggregate(
    pipeline
```

```
)
result = list(result)
```

In [145…
```
#
# Execute this cell to validate your query.
#
len(result)
```

Out[145… 326

In [146…
```
#
# Execute this cell to validate and display your results.
#
result[0:20]
```

Out[146…
```
[{'orderNumber': 10100, 'totalValue': 10223.83},
 {'orderNumber': 10101, 'totalValue': 10549.01},
 {'orderNumber': 10102, 'totalValue': 5494.78},
 {'orderNumber': 10103, 'totalValue': 50218.95},
 {'orderNumber': 10104, 'totalValue': 40206.2},
 {'orderNumber': 10105, 'totalValue': 53959.21},
 {'orderNumber': 10106, 'totalValue': 52151.81},
 {'orderNumber': 10107, 'totalValue': 22292.62},
 {'orderNumber': 10108, 'totalValue': 51001.22},
 {'orderNumber': 10109, 'totalValue': 25833.14},
 {'orderNumber': 10110, 'totalValue': 48425.69},
 {'orderNumber': 10111, 'totalValue': 16537.85},
 {'orderNumber': 10112, 'totalValue': 7674.94},
 {'orderNumber': 10113, 'totalValue': 11044.3},
 {'orderNumber': 10114, 'totalValue': 33383.14},
 {'orderNumber': 10115, 'totalValue': 21665.98},
 {'orderNumber': 10116, 'totalValue': 1627.56},
 {'orderNumber': 10117, 'totalValue': 44380.15},
 {'orderNumber': 10118, 'totalValue': 3101.4},
 {'orderNumber': 10119, 'totalValue': 35826.33}]
```

Neo4j

Use Neo4j and the sample Movie Data for these queries.

N1

Write a query that returns the names of people who directed movies and the title of the movies they directed.

In [39]:
```
q = '''
MATCH (p:Person)-[:DIRECTED]->(m:Movie)
RETURN p.name AS director, m.title AS movie
ORDER BY director, movie;
'''
```

In [40]:
```
run_query(q)
```

```
Result is
{'director': 'Cameron Crowe', 'movie': 'Jerry Maguire'}
{'director': 'Chris Columbus', 'movie': 'Bicentennial Man'}
{'director': 'Clint Eastwood', 'movie': 'Unforgiven'}
{'director': 'Danny DeVito', 'movie': 'Hoffa'}
{'director': 'Frank Darabont', 'movie': 'The Green Mile'}
{'director': 'Howard Deutch', 'movie': 'The Replacements'}
{'director': 'James L. Brooks', 'movie': 'As Good as It Gets'}
{'director': 'James Marshall', 'movie': 'Ninja Assassin'}
{'director': 'James Marshall', 'movie': 'V for Vendetta'}
{'director': 'Jan de Bont', 'movie': 'Twister'}
{'director': 'John Patrick Stanley', 'movie': 'Joe Versus the Volcano'}
{'director': 'Lana Wachowski', 'movie': 'Cloud Atlas'}
{'director': 'Lana Wachowski', 'movie': 'Speed Racer'}
{'director': 'Lana Wachowski', 'movie': 'The Matrix'}
{'director': 'Lana Wachowski', 'movie': 'The Matrix Reloaded'}
{'director': 'Lana Wachowski', 'movie': 'The Matrix Revolutions'}
{'director': 'Lilly Wachowski', 'movie': 'Cloud Atlas'}
{'director': 'Lilly Wachowski', 'movie': 'Speed Racer'}
{'director': 'Lilly Wachowski', 'movie': 'The Matrix'}
{'director': 'Lilly Wachowski', 'movie': 'The Matrix Reloaded'}
{'director': 'Lilly Wachowski', 'movie': 'The Matrix Revolutions'}
{'director': 'Mike Nichols', 'movie': "Charlie Wilson's War"}
{'director': 'Mike Nichols', 'movie': 'The Birdcage'}
{'director': 'Milos Forman', 'movie': "One Flew Over the Cuckoo's Nest"}
{'director': 'Nancy Meyers', 'movie': "Something's Gotta Give"}
{'director': 'Nora Ephron', 'movie': 'Sleepless in Seattle'}
{'director': 'Nora Ephron', 'movie': "You've Got Mail"}
{'director': 'Penny Marshall', 'movie': 'A League of Their Own'}
{'director': 'Rob Reiner', 'movie': 'A Few Good Men'}
{'director': 'Rob Reiner', 'movie': 'Stand By Me'}
{'director': 'Rob Reiner', 'movie': 'When Harry Met Sally'}
{'director': 'Robert Longo', 'movie': 'Johnny Mnemonic'}
{'director': 'Robert Zemeckis', 'movie': 'Cast Away'}
{'director': 'Robert Zemeckis', 'movie': 'The Polar Express'}
{'director': 'Ron Howard', 'movie': 'Apollo 13'}
{'director': 'Ron Howard', 'movie': 'Frost/Nixon'}
{'director': 'Ron Howard', 'movie': 'The Da Vinci Code'}
{'director': 'Scott Hicks', 'movie': 'Snow Falling on Cedars'}
{'director': 'Taylor Hackford', 'movie': "The Devil's Advocate"}
{'director': 'Tom Hanks', 'movie': 'That Thing You Do'}
{'director': 'Tom Tykwer', 'movie': 'Cloud Atlas'}
{'director': 'Tony Scott', 'movie': 'Top Gun'}
{'director': 'Vincent Ward', 'movie': 'What Dreams May Come'}
{'director': 'Werner Herzog', 'movie': 'RescueDawn'}
```

N2

Write a query that computes the director_name, movie_title, and actor_name for each movie and order by director name.

In [41]:
```
q = """
MATCH (d:Person)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(a:Person)
RETURN d.name AS director_name, m.title AS movie_title, a.name AS actor_name
ORDER BY director_name
"""
```

In [42]:
```
run_query(q)
```

```
Result is
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Tom Cruise'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Cuba Gooding Jr.'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Renee Zellweger'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Kelly Preston'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': "Jerry O'Connell"}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Jay Mohr'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Bonnie Hunt'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Regina King'}
{'director_name': 'Cameron Crowe', 'movie_title': 'Jerry Maguire', 'actor_name': 'Jonathan Lipnicki'}
{'director_name': 'Chris Columbus', 'movie_title': 'Bicentennial Man', 'actor_name': 'Robin Williams'}
{'director_name': 'Chris Columbus', 'movie_title': 'Bicentennial Man', 'actor_name': 'Oliver Platt'}
{'director_name': 'Clint Eastwood', 'movie_title': 'Unforgiven', 'actor_name': 'Gene Hackman'}
{'director_name': 'Clint Eastwood', 'movie_title': 'Unforgiven', 'actor_name': 'Richard Harris'}
{'director_name': 'Clint Eastwood', 'movie_title': 'Unforgiven', 'actor_name': 'Clint Eastwood'}
{'director_name': 'Danny DeVito', 'movie_title': 'Hoffa', 'actor_name': 'Jack Nicholson'}
{'director_name': 'Danny DeVito', 'movie_title': 'Hoffa', 'actor_name': 'J.T. Walsh'}
{'director_name': 'Danny DeVito', 'movie_title': 'Hoffa', 'actor_name': 'Danny DeVito'}
{'director_name': 'Danny DeVito', 'movie_title': 'Hoffa', 'actor_name': 'John C. Reilly'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Bonnie Hunt'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'James Cromwell'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Tom Hanks'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Michael Clarke Duncan'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'David Morse'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Sam Rockwell'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Gary Sinise'}
{'director_name': 'Frank Darabont', 'movie_title': 'The Green Mile', 'actor_name': 'Patricia Clarkson'}
{'director_name': 'Howard Deutch', 'movie_title': 'The Replacements', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Howard Deutch', 'movie_title': 'The Replacements', 'actor_name': 'Brooke Langton'}
{'director_name': 'Howard Deutch', 'movie_title': 'The Replacements', 'actor_name': 'Gene Hackman'}
{'director_name': 'Howard Deutch', 'movie_title': 'The Replacements', 'actor_name': 'Orlando Jones'}
{'director_name': 'James L. Brooks', 'movie_title': 'As Good as It Gets', 'actor_name': 'Jack Nicholson'}
{'director_name': 'James L. Brooks', 'movie_title': 'As Good as It Gets', 'actor_name': 'Cuba Gooding Jr.'}
{'director_name': 'James L. Brooks', 'movie_title': 'As Good as It Gets', 'actor_name': 'Helen Hunt'}
{'director_name': 'James L. Brooks', 'movie_title': 'As Good as It Gets', 'actor_name': 'Greg Kinnear'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_name': 'Hugo Weaving'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_name': 'Natalie Portman'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_name': 'Stephen Rea'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_name': 'John Hurt'}
{'director_name': 'James Marshall', 'movie_title': 'V for Vendetta', 'actor_name': 'Ben Miles'}
{'director_name': 'James Marshall', 'movie_title': 'Ninja Assassin', 'actor_name': 'Rick Yune'}
{'director_name': 'James Marshall', 'movie_title': 'Ninja Assassin', 'actor_name': 'Ben Miles'}
{'director_name': 'James Marshall', 'movie_title': 'Ninja Assassin', 'actor_name': 'Rain'}
{'director_name': 'James Marshall', 'movie_title': 'Ninja Assassin', 'actor_name': 'Naomie Harris'}
{'director_name': 'Jan de Bont', 'movie_title': 'Twister', 'actor_name': 'Helen Hunt'}
{'director_name': 'Jan de Bont', 'movie_title': 'Twister', 'actor_name': 'Zach Grenier'}
{'director_name': 'Jan de Bont', 'movie_title': 'Twister', 'actor_name': 'Bill Paxton'}
{'director_name': 'Jan de Bont', 'movie_title': 'Twister', 'actor_name': 'Philip Seymour Hoffman'}
{'director_name': 'John Patrick Stanley', 'movie_title': 'Joe Versus the Volcano', 'actor_name': 'Meg Rya
n'}
{'director_name': 'John Patrick Stanley', 'movie_title': 'Joe Versus the Volcano', 'actor_name': 'Tom Hank
s'}
{'director_name': 'John Patrick Stanley', 'movie_title': 'Joe Versus the Volcano', 'actor_name': 'Nathan La
ne'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Carrie-Anne Moss'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Laurence Fishburne'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Emil Eifrem'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Carrie-Anne Moss'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Laurence Fishburn
e'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Carrie-Anne Mos
s'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Laurence Fishbu
rne'}
{'director_name': 'Lana Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Tom Hanks'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Halle Berry'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Jim Broadbent'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Ben Miles'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Emile Hirsch'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'John Goodman'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Susan Sarandon'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Matthew Fox'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Christina Ricci'}
{'director_name': 'Lana Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Rain'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Carrie-Anne Moss'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Laurence Fishburne'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix', 'actor_name': 'Emil Eifrem'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Carrie-Anne Mos
s'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Laurence Fishburn
```

```
e'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Reloaded', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Carrie-Anne Mo
ss'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Laurence Fishb
urne'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'The Matrix Revolutions', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Tom Hanks'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Halle Berry'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Cloud Atlas', 'actor_name': 'Jim Broadbent'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Ben Miles'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Emile Hirsch'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'John Goodman'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Susan Sarandon'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Matthew Fox'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Christina Ricci'}
{'director_name': 'Lilly Wachowski', 'movie_title': 'Speed Racer', 'actor_name': 'Rain'}
{'director_name': 'Mike Nichols', 'movie_title': 'The Birdcage', 'actor_name': 'Robin Williams'}
{'director_name': 'Mike Nichols', 'movie_title': 'The Birdcage', 'actor_name': 'Nathan Lane'}
{'director_name': 'Mike Nichols', 'movie_title': 'The Birdcage', 'actor_name': 'Gene Hackman'}
{'director_name': 'Mike Nichols', 'movie_title': "Charlie Wilson's War", 'actor_name': 'Tom Hanks'}
{'director_name': 'Mike Nichols', 'movie_title': "Charlie Wilson's War", 'actor_name': 'Philip Seymour Hoff
man'}
{'director_name': 'Mike Nichols', 'movie_title': "Charlie Wilson's War", 'actor_name': 'Julia Roberts'}
{'director_name': 'Milos Forman', 'movie_title': "One Flew Over the Cuckoo's Nest", 'actor_name': 'Jack Nic
holson'}
{'director_name': 'Milos Forman', 'movie_title': "One Flew Over the Cuckoo's Nest", 'actor_name': 'Danny De
Vito'}
{'director_name': 'Nancy Meyers', 'movie_title': "Something's Gotta Give", 'actor_name': 'Keanu Reeves'}
{'director_name': 'Nancy Meyers', 'movie_title': "Something's Gotta Give", 'actor_name': 'Jack Nicholson'}
{'director_name': 'Nancy Meyers', 'movie_title': "Something's Gotta Give", 'actor_name': 'Diane Keaton'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Meg Ryan'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Greg Kinnear'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Tom Hanks'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Parker Posey'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Dave Chappelle'}
{'director_name': 'Nora Ephron', 'movie_title': "You've Got Mail", 'actor_name': 'Steve Zahn'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Meg Ryan'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Tom Hanks'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Rita Wilson'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Bill Pullman'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': 'Victor Garber'}
{'director_name': 'Nora Ephron', 'movie_title': 'Sleepless in Seattle', 'actor_name': "Rosie O'Donnell"}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Tom Hanks'}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': "Rosie O'Donnel
l"}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Bill Paxton'}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Madonna'}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Geena Davis'}
{'director_name': 'Penny Marshall', 'movie_title': 'A League of Their Own', 'actor_name': 'Lori Petty'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Tom Cruise'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Jack Nicholson'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Demi Moore'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Kevin Bacon'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Kiefer Sutherland'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Noah Wyle'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Cuba Gooding Jr.'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Kevin Pollak'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'J.T. Walsh'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'James Marshall'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Christopher Guest'}
{'director_name': 'Rob Reiner', 'movie_title': 'A Few Good Men', 'actor_name': 'Aaron Sorkin'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name': 'Kiefer Sutherland'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name': "Jerry O'Connell"}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name': 'River Phoenix'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name': 'Corey Feldman'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name': 'Wil Wheaton'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name': 'John Cusack'}
{'director_name': 'Rob Reiner', 'movie_title': 'Stand By Me', 'actor_name': 'Marshall Bell'}
{'director_name': 'Rob Reiner', 'movie_title': 'When Harry Met Sally', 'actor_name': 'Meg Ryan'}
{'director_name': 'Rob Reiner', 'movie_title': 'When Harry Met Sally', 'actor_name': 'Billy Crystal'}
{'director_name': 'Rob Reiner', 'movie_title': 'When Harry Met Sally', 'actor_name': 'Carrie Fisher'}
{'director_name': 'Rob Reiner', 'movie_title': 'When Harry Met Sally', 'actor_name': 'Bruno Kirby'}
{'director_name': 'Robert Longo', 'movie_title': 'Johnny Mnemonic', 'actor_name': 'Keanu Reeves'}
{'director_name': 'Robert Longo', 'movie_title': 'Johnny Mnemonic', 'actor_name': 'Takeshi Kitano'}
{'director_name': 'Robert Longo', 'movie_title': 'Johnny Mnemonic', 'actor_name': 'Dina Meyer'}
{'director_name': 'Robert Longo', 'movie_title': 'Johnny Mnemonic', 'actor_name': 'Ice-T'}
{'director_name': 'Robert Zemeckis', 'movie_title': 'Cast Away', 'actor_name': 'Helen Hunt'}
{'director_name': 'Robert Zemeckis', 'movie_title': 'Cast Away', 'actor_name': 'Tom Hanks'}
{'director_name': 'Robert Zemeckis', 'movie_title': 'The Polar Express', 'actor_name': 'Tom Hanks'}
{'director_name': 'Ron Howard', 'movie_title': 'The Da Vinci Code', 'actor_name': 'Tom Hanks'}
{'director_name': 'Ron Howard', 'movie_title': 'The Da Vinci Code', 'actor_name': 'Ian McKellen'}
{'director_name': 'Ron Howard', 'movie_title': 'The Da Vinci Code', 'actor_name': 'Audrey Tautou'}
{'director_name': 'Ron Howard', 'movie_title': 'The Da Vinci Code', 'actor_name': 'Paul Bettany'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name': 'Kevin Bacon'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name': 'Sam Rockwell'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name': 'Frank Langella'}
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name': 'Michael Sheen'}
```

```
{'director_name': 'Ron Howard', 'movie_title': 'Frost/Nixon', 'actor_name': 'Oliver Platt'}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Kevin Bacon'}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Tom Hanks'}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Gary Sinise'}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Ed Harris'}
{'director_name': 'Ron Howard', 'movie_title': 'Apollo 13', 'actor_name': 'Bill Paxton'}
{'director_name': 'Scott Hicks', 'movie_title': 'Snow Falling on Cedars', 'actor_name': 'Max von Sydow'}
{'director_name': 'Scott Hicks', 'movie_title': 'Snow Falling on Cedars', 'actor_name': 'Ethan Hawke'}
{'director_name': 'Scott Hicks', 'movie_title': 'Snow Falling on Cedars', 'actor_name': 'Rick Yune'}
{'director_name': 'Scott Hicks', 'movie_title': 'Snow Falling on Cedars', 'actor_name': 'James Cromwell'}
{'director_name': 'Taylor Hackford', 'movie_title': "The Devil's Advocate", 'actor_name': 'Keanu Reeves'}
{'director_name': 'Taylor Hackford', 'movie_title': "The Devil's Advocate", 'actor_name': 'Charlize Thero
n'}
{'director_name': 'Taylor Hackford', 'movie_title': "The Devil's Advocate", 'actor_name': 'Al Pacino'}
{'director_name': 'Tom Hanks', 'movie_title': 'That Thing You Do', 'actor_name': 'Charlize Theron'}
{'director_name': 'Tom Hanks', 'movie_title': 'That Thing You Do', 'actor_name': 'Tom Hanks'}
{'director_name': 'Tom Hanks', 'movie_title': 'That Thing You Do', 'actor_name': 'Liv Tyler'}
{'director_name': 'Tom Tykwer', 'movie_title': 'Cloud Atlas', 'actor_name': 'Hugo Weaving'}
{'director_name': 'Tom Tykwer', 'movie_title': 'Cloud Atlas', 'actor_name': 'Tom Hanks'}
{'director_name': 'Tom Tykwer', 'movie_title': 'Cloud Atlas', 'actor_name': 'Halle Berry'}
{'director_name': 'Tom Tykwer', 'movie_title': 'Cloud Atlas', 'actor_name': 'Jim Broadbent'}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Tom Cruise'}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Kelly McGillis'}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Val Kilmer'}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Anthony Edwards'}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Tom Skerritt'}
{'director_name': 'Tony Scott', 'movie_title': 'Top Gun', 'actor_name': 'Meg Ryan'}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'actor_name': 'Cuba Gooding Jr.'}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'actor_name': 'Annabella Sciorra'}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'actor_name': 'Max von Sydow'}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'actor_name': 'Werner Herzog'}
{'director_name': 'Vincent Ward', 'movie_title': 'What Dreams May Come', 'actor_name': 'Robin Williams'}
{'director_name': 'Werner Herzog', 'movie_title': 'RescueDawn', 'actor_name': 'Marshall Bell'}
{'director_name': 'Werner Herzog', 'movie_title': 'RescueDawn', 'actor_name': 'Steve Zahn'}
{'director_name': 'Werner Herzog', 'movie_title': 'RescueDawn', 'actor_name': 'Christian Bale'}
{'director_name': 'Werner Herzog', 'movie_title': 'RescueDawn', 'actor_name': 'Zach Grenier'}
```

N3

Return a list of all people who acted in a movie with Tom Hanks. Order the list by movie.title.

In [43]:
```
q = """
MATCH (tom:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(coactor:Person)
RETURN m.title AS movie_title, coactor.name AS actor_name
ORDER BY m.title
"""
```

In [44]:
```
run_query(q)
```

```
Result is
{'movie_title': 'A League of Their Own', 'actor_name': "Rosie O'Donnell"}
{'movie_title': 'A League of Their Own', 'actor_name': 'Bill Paxton'}
{'movie_title': 'A League of Their Own', 'actor_name': 'Madonna'}
{'movie_title': 'A League of Their Own', 'actor_name': 'Geena Davis'}
{'movie_title': 'A League of Their Own', 'actor_name': 'Lori Petty'}
{'movie_title': 'Apollo 13', 'actor_name': 'Kevin Bacon'}
{'movie_title': 'Apollo 13', 'actor_name': 'Gary Sinise'}
{'movie_title': 'Apollo 13', 'actor_name': 'Ed Harris'}
{'movie_title': 'Apollo 13', 'actor_name': 'Bill Paxton'}
{'movie_title': 'Cast Away', 'actor_name': 'Helen Hunt'}
{'movie_title': "Charlie Wilson's War", 'actor_name': 'Philip Seymour Hoffman'}
{'movie_title': "Charlie Wilson's War", 'actor_name': 'Julia Roberts'}
{'movie_title': 'Cloud Atlas', 'actor_name': 'Hugo Weaving'}
{'movie_title': 'Cloud Atlas', 'actor_name': 'Halle Berry'}
{'movie_title': 'Cloud Atlas', 'actor_name': 'Jim Broadbent'}
{'movie_title': 'Joe Versus the Volcano', 'actor_name': 'Meg Ryan'}
{'movie_title': 'Joe Versus the Volcano', 'actor_name': 'Nathan Lane'}
{'movie_title': 'Sleepless in Seattle', 'actor_name': 'Meg Ryan'}
{'movie_title': 'Sleepless in Seattle', 'actor_name': 'Rita Wilson'}
{'movie_title': 'Sleepless in Seattle', 'actor_name': 'Bill Pullman'}
{'movie_title': 'Sleepless in Seattle', 'actor_name': 'Victor Garber'}
{'movie_title': 'Sleepless in Seattle', 'actor_name': "Rosie O'Donnell"}
{'movie_title': 'That Thing You Do', 'actor_name': 'Charlize Theron'}
{'movie_title': 'That Thing You Do', 'actor_name': 'Liv Tyler'}
{'movie_title': 'The Da Vinci Code', 'actor_name': 'Ian McKellen'}
{'movie_title': 'The Da Vinci Code', 'actor_name': 'Audrey Tautou'}
{'movie_title': 'The Da Vinci Code', 'actor_name': 'Paul Bettany'}
{'movie_title': 'The Green Mile', 'actor_name': 'Bonnie Hunt'}
{'movie_title': 'The Green Mile', 'actor_name': 'James Cromwell'}
{'movie_title': 'The Green Mile', 'actor_name': 'Michael Clarke Duncan'}
{'movie_title': 'The Green Mile', 'actor_name': 'David Morse'}
{'movie_title': 'The Green Mile', 'actor_name': 'Sam Rockwell'}
{'movie_title': 'The Green Mile', 'actor_name': 'Gary Sinise'}
{'movie_title': 'The Green Mile', 'actor_name': 'Patricia Clarkson'}
{'movie_title': "You've Got Mail", 'actor_name': 'Meg Ryan'}
{'movie_title': "You've Got Mail", 'actor_name': 'Greg Kinnear'}
{'movie_title': "You've Got Mail", 'actor_name': 'Parker Posey'}
{'movie_title': "You've Got Mail", 'actor_name': 'Dave Chappelle'}
{'movie_title': "You've Got Mail", 'actor_name': 'Steve Zahn'}
```

In [ ]:

In [ ]: